

Maximum Occurring Character

Problem Statement

Given a string `s`, find the character that occurs the **maximum number of times**.

If multiple characters have the same maximum frequency, return the character that comes **first in alphabetical/lexicographical order**.

Example 1

Input

```
s = "testsample"
```

Output

```
e
```

Explanation

Character frequencies:

```
t → 2  
e → 2  
s → 2  
a → 1  
m → 1  
p → 1  
l → 1
```

The maximum frequency is `2`.

Among `e`, `s`, and `t`, the smallest character is:

```
e
```

Therefore:

```
Output = 'e'
```

Example 2

Input

```
s = "output"
```

Output

```
t
```

Explanation

Frequency:

```
o → 2  
t → 2  
u → 1  
p → 1
```

Both `o` and `t` occur twice.

Since:

```
'o' < 't'
```

the answer is:

```
'o'
```

The exact expected output depends on the problem's tie-breaking rule. This implementation specifically chooses the **smaller character** when frequencies are equal.

Approach

The solution uses a `HashMap<Character, Integer>` to store the frequency of every character.

The approach has two main steps:

Step 1

Count the frequency of every character.

character → frequency

Step 2

Find the character with the highest frequency.

If two characters have the same frequency, choose the smaller character.

Java Implementation

```
class Solution {
    public static char getMaxOccuringChar(String s) {

        char[] arr = s.toCharArray();

        HashMap<Character, Integer> map = new HashMap<>();

        for (char num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        char max = arr[0];
        int maxcount = 0;

        for (char key : map.keySet()) {

            if (map.get(key) > maxcount) {
                maxcount = map.get(key);
                max = key;
            }

            else if (map.get(key) == maxcount && key < max) {
```

```
        max = key;
    }
}

return max;
}
```

Code Explanation

1. Convert String to Character Array

```
char[] arr = s.toCharArray();
```

For:

```
s = "banana"
```

we get:

```
arr = ['b', 'a', 'n', 'a', 'n', 'a']
```

2. Create HashMap

```
HashMap<Character, Integer> map = new HashMap<>();
```

The HashMap stores:

```
Character → Number of occurrences
```

For "banana":

```
b → 1  
a → 3  
n → 2
```

3. Count Character Frequencies

```
for (char num : arr) {  
    map.put(num, map.getOrDefault(num, 0) + 1);  
}
```

This is the main frequency-counting logic.

Understanding `getOrDefault()`

```
map.getOrDefault(num, 0)
```

means:

If the character already exists in the map, get its current count. Otherwise, use `0`.

Then:

```
+ 1
```

increases the count.

Example

For:

```
"banana"
```

First character: `b`

```
map.getOrDefault('b', 0)  
= 0
```

Then:

$$0 + 1 = 1$$

Map:

$$b \rightarrow 1$$

Next character: **a**

$$a \rightarrow 1$$

Next character: **n**

$$n \rightarrow 1$$

Next character: **a**

a already exists:

$$a \rightarrow 1$$

So:

$$1 + 1 = 2$$

Now:

$$a \rightarrow 2$$

Eventually:

$$\begin{array}{l} b \rightarrow 1 \\ a \rightarrow 3 \\ n \rightarrow 2 \end{array}$$

4. Initialize Maximum Character

```
char max = arr[0];
```

We initially assume that the first character is the maximum occurring character.

For:

```
s = "banana"
```

```
max = 'b'
```

5. Initialize Maximum Count

```
int maxcount = 0;
```

This stores the highest frequency found so far.

Initially:

```
maxcount = 0
```

6. Traverse the HashMap

```
for (char key : map.keySet())
```

`map.keySet()` gives all unique characters.

For:

```
banana
```

the keys are:

```
b, a, n
```

The order of a `HashMap` is **not guaranteed**.

That's why the tie-breaking condition is important.

7. Check for Higher Frequency

```
if (map.get(key) > maxcount)
```

If the current character occurs more times than the current maximum, update both:

```
maxcount = map.get(key);  
max = key;
```

For example:

```
current character = 'a'  
frequency = 3  
maxcount = 1
```

Since:

```
3 > 1
```

update:

```
maxcount = 3  
max = 'a'
```

8. Handle Equal Frequencies

This is the most important part:


```
else if (map.get(key) == maxcount && key < max) {  
    max = key;  
}
```

This handles the situation where two characters have the same frequency.

Suppose:

```
a → 3  
b → 3
```

Both have the same maximum frequency.

We want the smaller character.

Compare:

```
'a' < 'b'
```

This is true, so:

```
max = 'a';
```

Why `key < max`?

Java compares `char` values using their Unicode values.

For lowercase English letters:

```
'a' < 'b' < 'c' < ... < 'z'
```

For example:

```
'a' = 97  
'b' = 98  
'c' = 99
```

Therefore:

```
key < max
```

means:

Is the current character alphabetically smaller than the current maximum character?

Dry Run

Consider:

```
s = "banana"
```

Frequency map:

```
b → 1  
a → 3  
n → 2
```

Initial:

```
max = 'b'  
maxcount = 0
```

Character **b**

```
frequency = 1
```

Check:

```
1 > 0
```

True.

Update:

```
max = 'b'  
maxcount = 1
```

Character a

```
frequency = 3
```

Check:

```
3 > 1
```

True.

Update:

```
max = 'a'  
maxcount = 3
```

Character n

```
frequency = 2
```

Check:

```
2 > 3
```

False.

Tie?

```
2 == 3
```

False.

No update.

Final:

```
max = 'a'  
maxcount = 3
```

Therefore:

```
Output = 'a'
```

Tie Case Dry Run

Consider:

```
s = "aabbcc"
```

Frequency:

```
a → 2  
b → 2  
c → 2
```

Suppose current values eventually produce:

```
max = 'c'  
maxcount = 2
```

Then when we encounter:

```
key = 'a'
```

we check:

```
map.get(key) == maxcount
```

which gives:

```
2 == 2
```

True.

Then:

```
key < max
```

becomes:

```
'a' < 'c'
```

True.

So:

```
max = 'a'
```

Thus the smallest character is selected.

Why HashMap?

A `HashMap` is useful because it stores a mapping:

```
Character → Frequency
```

Example:

```
'a' → 4  
'b' → 2  
'c' → 7
```

The average time for:

```
map.get()  
map.put()  
map.getOrDefault()
```

is `O(1)`.

Important Point About HashMap

A `HashMap` does **not guarantee insertion order**.

For example, even if characters are inserted as:

```
a  
b  
c
```

the `keySet()` traversal order is not guaranteed to be:

```
a → b → c
```

Therefore, you correctly handle ties explicitly with:

```
else if (map.get(key) == maxcount && key < max)
```

This ensures the result does not depend on HashMap iteration order.

Complete Dry Run Table

For:

```
s = "banana"
```

Character	Frequency	<code>maxcount</code>	Before	Action	<code>max</code>
<code>b</code>	1		0	Update	<code>b</code>
<code>a</code>	3		1	Update	<code>a</code>

Character	Frequency	maxcount	Before	Action	max
n	2		3	No update	a

Final:

```
max = 'a'
```

Edge Cases

1. Single Character

Input:

"a"

Output:

"a"

2. All Characters Different

Input:

"abc"

All frequencies are 1.

The smallest character is:

'a'

Output:

'a'

3. Repeated Character

Input:
"aaaa"

Frequency:

a → 4

Output:

'a'

4. Tie in Frequency

Input:
"bbccaa"

Frequencies:

a → 2
b → 2
c → 2

Smallest character:

'a'

Output:

'a'

Alternative Approach

If the problem is guaranteed to contain only lowercase English letters, we can use an integer array instead of a HashMap.

```
int[] freq = new int[26];

for (char ch : s.toCharArray()) {
    freq[ch - 'a']++;
}
```

Then iterate from `a` to `z`.

This can be simpler and uses constant extra space.

However, your `HashMap` solution is more general because it can handle a wider range of characters.

Complexity Analysis

Let `n` be the length of the string and `k` be the number of unique characters.

Building the frequency map

$O(n)$

because every character is processed once.

Finding maximum

$O(k)$

where:

$k \leq n$

Therefore:

Overall Time Complexity = $O(n)$

Space Complexity

The HashMap stores up to k unique characters:

Space Complexity = $O(k)$

In the worst case:

$O(n)$

DSA Concepts Used

- String
- Character array
- HashMap
- Frequency counting
- `getOrDefault()`
- `keySet()`
- Character comparison
- Tie-breaking logic

Key Takeaway

The main pattern is:

```
String
  ↓
Count frequency using HashMap
  ↓
Find maximum frequency
  ↓
If frequencies are equal
  ↓
Choose smaller character
```

The key condition is:

```
else if (map.get(key) == maxcount && key < max)
```

It means:

If the current character has the same maximum frequency, choose it if it is alphabetically smaller than the current answer.

Final Complexity

Time Complexity : $O(n)$

Space Complexity : $O(k)$, worst case $O(n)$

Core pattern: Frequency Map → Maximum Frequency → Lexicographical Tie Breaking.